

Especificación Funcional Formal

Editor de Mundos – Simulador LEGO EV3 (Pybricks)

1. Alcance

Define requisitos funcionales (RF), no funcionales (RNF), historias de usuario, criterios de aceptación y contratos de software para el módulo Editor de Mundos, alineado con SDD.

2. Actores

- Usuario (docente/estudiante)
 - Sistema (Editor)
 - Motor de Simulación
-

3. Requisitos Funcionales (RF)

RF-01 Crear mundo - Descripción: Inicializa un world vacío con grid. - Entrada: dimensiones (opcional) - Salida: WorldModel vacío

RF-02 Seleccionar asset - Entrada: tipo de asset - Salida: estado de selección activo

RF-03 Colocar asset - Entrada: (x,y) del mouse - Proceso: snap a grid, validación - Salida: Placement creado

RF-04 Eliminar asset - Entrada: selección/posición - Salida: asset removido

RF-05 Mover asset - Entrada: asset_id, nueva posición - Salida: asset reubicado (validado)

RF-06 Guardar mundo - Salida: JSON versionado

RF-07 Cargar mundo - Entrada: JSON - Salida: WorldModel reconstruido

RF-08 Validar mundo - Salida: lista de errores/advertencias

RF-09 Conectividad de líneas - Verifica continuidad de trazas

4. Requisitos No Funcionales (RNF)

RNF-01 Determinismo: misma spec $\Rightarrow$ mismo world. **RNF-02 Rendimiento:** <16 ms por interacción. **RNF-03 Portabilidad:** desktop (Tkinter) y web (Canvas). **RNF-04 Escalabilidad:** $\geq 10k$ placements. **RNF-05 Usabilidad:** feedback inmediato de validación.

5. Historias de Usuario

HU-01 Como usuario quiero crear un mundo vacío para iniciar diseño. - Criterios: grid visible, sin assets.

HU-02 Como usuario quiero colocar líneas para definir trayectorias. - Criterios: snap correcto, sin solapamientos inválidos.

HU-03 Como usuario quiero guardar mi mundo. - Criterios: JSON válido, reproducible.

HU-04 Como usuario quiero validar el mundo. - Criterios: reporte de errores claro.

6. Criterios de Aceptación (Ejemplos)

- CA-01: $x \% 32 == 0$ y $y \% 32 == 0$
 - CA-02: no solapamiento inválido
 - CA-03: líneas conectadas o error
-

7. Modelo de Datos

```
from dataclasses import dataclass
from typing import List, Literal

AssetType = Literal["robot", "line", "wall", "zone", "floor"]

@dataclass
class Placement:
    id: str
    type: AssetType
    x: int
    y: int
    rotation: int = 0

@dataclass
class WorldModel:
    grid_size: int
    placements: List[Placement]
```

8. Contratos (Interfaces)

```
class EditorAPI:
    def create_world(self, grid_size: int = 32) -> WorldModel: ...
```

```

def place_asset(self, world: WorldModel, p: Placement) -> None: ...

def remove_asset(self, world: WorldModel, asset_id: str) -> None: ...

def move_asset(self, world: WorldModel, asset_id: str, x: int, y: int) -
> None: ...

def validate(self, world: WorldModel) -> list[str]: ...

def save(self, world: WorldModel) -> str: ...

def load(self, data: str) -> WorldModel: ...

```

Precondiciones

- Coordenadas alineadas a grid
- type válido

Postcondiciones

- World consistente

9. Reglas de Validación

- Alineación: $x \% \text{grid} == 0$
- Límites del canvas
- Conectividad de líneas (grafo implícito)
- No solapamiento incompatible

10. Persistencia

```

{
  "version": 1,
  "grid_size": 32,
  "placements": []
}

```

11. Pruebas (derivadas de specs)

```

def test_snap_to_grid():
    assert (65 // 32) * 32 == 64

```

```
def test_alignment():  
    assert 64 % 32 == 0
```

12. Integración con Simulación

```
def load_world(world: WorldModel):  
    # traduce a SimulationState  
    pass
```

13. Trazabilidad (SDD)

• RF → HU → CA → Test

Ejemplo: RF-03 → HU-02 → CA-01 → test_snap_to_grid

14. Roadmap de Implementación

1. Core model (WorldModel)
 2. Grid & snapping
 3. Placement engine
 4. Validation engine
 5. Persistence
 6. UI integration
-

15. Cierre

Este documento convierte las especificaciones en artefactos directamente implementables, alineando SDD con desarrollo modular, testeable y asistido por IA.